

OPEX too. This series of articles provides step-by-step guidance on how to integrate NFV with the DevOps process.

Since the experimental work involves many technologies, we plan to publish our work in eight parts that focus on distributed systems, containerization, Kubernetes, open source cloud computing, software-defined networking, NFV, DevOps methodologies, and NFV integration with DevOps.

The need today is for fast and powerful machines that can host our applications or store important documents, which should be available around the globe. This cannot be achieved via a monolithic system design. We need something that is robust and reliable, and available from our laptops, workstations, and mobile devices. This has led to a ‘distributed system’, first coined by Paul Baran, where the idea is to create a distributed communication network in which messages are sent through a network of switching nodes until they reach the desired destination.

The distributed system we see today is different from what was envisioned back then, but the basic concept remains the same. Such a system is basically a coalition of multiple machines that are viewed as one and come together to achieve a task.

Maarten van Steen and Andrew S. Tanenbaum in their book ‘Distributed Systems’, state: “A distributed system is a collection of autonomous computing elements that appears to its users as a single coherent system.”

This defines two important characteristics of such systems. The first is that these are collections of multiple computing elements, which can behave and work independently but are brought together. And the second is that they are viewed by users as a single unit, so the person using such a system does not differentiate between a single unit or multiple units. We can envision these systems as a graph (Figure 1) — each

computing unit is a node in that graph and each node is connected to another node via some network.

Architecture

By now we may look at distributed systems as a bunch of loosely interconnected processing devices working together over an underlying network, but there is more to this. The architectural styles define how these processing units are connected, and there

are several styles:

- Layered architectures
- Object-based architectures
- Resource-centered architectures
- Event-based architectures

Layered architectures: The idea behind layered architectures is to organise components of distributed systems in a layered manner. Here layer L_j makes requests to L_i where $(i < j)$, and expects a response in return (as shown in Figure 2).

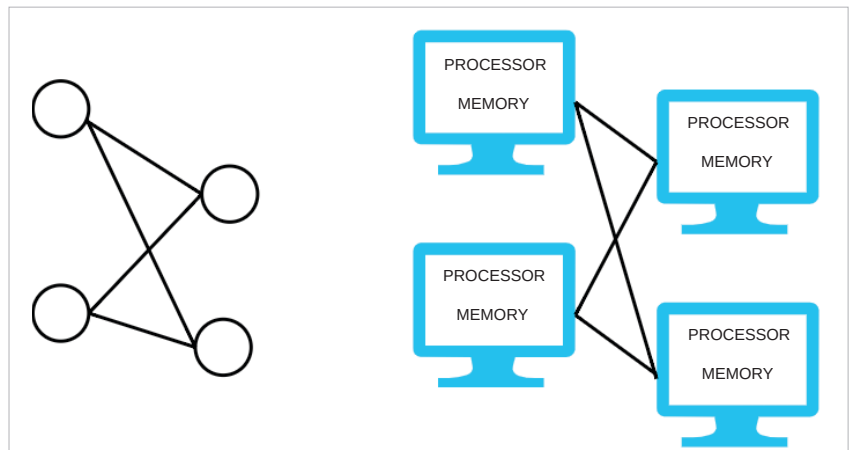


Figure 1: Node graph

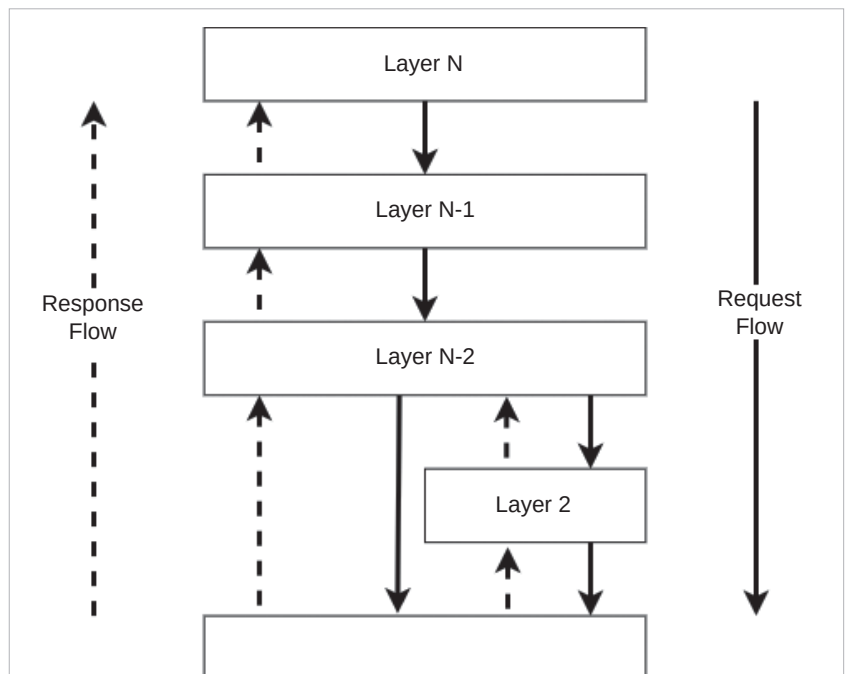


Figure 2: Layered organisation of various components